CÓDIGOCLARO · RUTA POO EN JAVA

Ruta POO en Java · Principios y diseño

15 temas · Java SE 17+ · ejemplos y prácticas con documentación enlazada

Las explicaciones se apoyan en documentación primaria; los ejemplos y prácticas se identifican como material didáctico del curso.

Tema 01 · Clases, objetos y estado

Objetivo

Distinguir la definición de una clase de cada objeto creado a partir de ella y localizar estado y comportamiento.

Explicación

Una clase declara miembros como campos, constructores y métodos. Un objeto es una instancia concreta: dos objetos de Libro comparten la misma estructura, pero cada uno mantiene su propio título e ISBN. Los métodos describen operaciones que se pueden pedir a la instancia.

En la ruta usaremos una biblioteca como dominio conductor. El modelo es didáctico: sus reglas (por ejemplo, el máximo de préstamos) se proponen como decisiones del ejercicio, no como políticas bibliotecarias universales.

Ejemplo guiado

Declarar una clase y crear una instancia · Java

```
class Libro {
  private final String isbn;
  private final String titulo;
  private boolean prestado;
  Libro(String isbn, String titulo) {
    this.isbn = isbn; this.titulo = titulo;
  }
  boolean estaPrestado() { return prestado; }
}
Libro manual = new Libro("978-1", "POO");
```

Lectura

Libro es la clase; manual es una referencia a un objeto concreto. isbn, titulo y prestado son estado de cada instancia; estaPrestado() expone una operación sobre ese estado.

Alcance y precisión

Es un fragmento Java: se han omitido paquete y clase ejecutable. El acceso private oculta campos; el objeto se crea con new y una llamada al constructor.

Práctica

Crea dos libros con ISBN distintos. Añade un método que cambie el estado de prestado y comprueba que cambiar un libro no altera el otro.

Comprobación

Cada instancia conserva sus propios campos de instancia. No uses static para datos que deban variar de un libro a otro.

Fuentes enlazadas: POO en Java · Dev.java · Clases y objetos · Crear objetos

Tema 02 · Encapsulación e invariantes

Objetivo

Proteger el estado interno y hacer que las operaciones públicas mantengan reglas coherentes del objeto.

Explicación

Encapsular no consiste solo en poner campos private y crear getters. También significa decidir qué operaciones ofrece el objeto y evitar que el resto del programa pueda dejarlo en un estado inválido mediante asignaciones arbitrarias.

En este ejemplo, el límite de tres préstamos es una regla inventada para practicar. El método registra el cambio o informa que no puede hacerlo; no expone un setter para que cualquier cliente altere el contador sin validar la regla.

Ejemplo guiado

Una operación que protege el estado · Java

```
class LimitePrestamos {
  private int activos;
  boolean registrar() {
    if (activos >= 3) return false;
    activos++;
    return true;
  }
  int total() { return activos; }
}
```

Lectura

El campo no se puede cambiar directamente desde código externo. registrar() concentra la comprobación y la modificación; false comunica que el límite del ejercicio ya se alcanzó.

Alcance y precisión

Los modificadores de acceso controlan qué código puede usar cada miembro. No sustituyen validación de entradas, autorización de usuarios ni controles de seguridad del sistema.

Práctica

Añade devolver(). Debe reducir el contador solo si hay un préstamo activo; define qué devuelve o comunica cuando no hay ninguno que devolver.

Comprobación

La regla debe seguir siendo cierta después de cada operación pública: activos nunca queda negativo ni supera tres.

Fuentes enlazadas: Control de acceso a miembros · Clases, estado y comportamiento

Tema 03 · Constructores e inicialización

Objetivo

Crear objetos en un estado inicial completo y comprender la sobrecarga y el constructor por defecto.

Explicación

En Java, un constructor tiene el nombre de la clase y no declara tipo de retorno. Se ejecuta al crear la instancia y puede recibir argumentos para inicializar sus campos. Varios constructores pueden coexistir si sus listas de parámetros son distintas.

Si una clase no declara ningún constructor, el compilador proporciona un constructor sin argumentos. En cuanto la clase declara uno, ese constructor por defecto ya no se añade. Diseña el constructor según los estados válidos que necesita el objeto.

Ejemplo guiado

Inicializar todos los datos necesarios · Java

```
import java.time.LocalDate;
import java.util.Objects;
class Prestamo {
  private final Libro libro;
  private final LocalDate inicio;
```

```
  Prestamo(Libro libro, LocalDate inicio) {
    this.libro = Objects.requireNonNull(libro);
    this.inicio = Objects.requireNonNull(inicio);
  }
}
```

Lectura

Los campos final se asignan en el constructor y no se reasignan después. requireNonNull falla de inmediato si falta un argumento requerido, en vez de dejar el préstamo parcialmente inicializado.

Alcance y precisión

Objects.requireNonNull lanza NullPointerException cuando recibe null. Esta precondición falla pronto; si el dominio exige otro contrato, elige y documenta otro error.

Práctica

Añade un segundo constructor que reciba un ISBN y resuelva el Libro con un repositorio. Decide si esa resolución pertenece al constructor o a un servicio externo.

Comprobación

Prueba la ruta normal y una entrada nula. Revisa qué constructores existen antes de instanciar la clase sin argumentos.

Fuentes enlazadas: Constructores en Dev.java · Crear y usar objetos · API Objects · Java SE 17

Tema 04 · Abstracción y clases abstractas

Objetivo

Separar la operación común de sus detalles y saber cuándo una clase abstracta expresa una base compartida.

Explicación

Una abstracción publica lo que el cliente necesita saber y deja los detalles detrás de un contrato. Una clase abstract puede combinar métodos implementados con métodos abstractos que las subclases concretas deben completar.

Una clase abstracta no se instancia directamente. Tiene sentido cuando los tipos relacionados comparten estado o implementación; si solo se necesita declarar una capacidad sin estado común, una interfaz puede encajar mejor.

Ejemplo guiado

Fijar un algoritmo y delegar un paso · Java

```
abstract class FormatoFicha {
  final String renderizar(Libro libro) {
    return encabezado() + cuerpo(libro);
  }
  abstract String encabezado();
  abstract String cuerpo(Libro libro);
}
class FichaBreve extends FormatoFicha {
  String encabezado() { return "LIBRO"; }
  String cuerpo(Libro l) { return l.titulo(); }
}
```

Lectura

La base define renderizar y deja dos pasos que cada formato decide. El fragmento presupone un método titulo() en Libro; el esqueleto omite detalles no relevantes para la abstracción.

Alcance y precisión

La clase abstracta puede contener campos no estáticos y métodos con acceso privado o protegido. Una clase Java solo puede extender una clase directa.

Práctica

Diseña dos formatos de ficha. Mantén en la base solo el flujo que realmente comparten; si no comparten estado ni implementación, prueba la alternativa con interfaz.

Comprobación

No marques abstracta una clase solo para impedir new: pregunta qué contrato o implementación común representa y quién completará cada operación.

Fuentes enlazadas: Clases abstractas · Dev.java · Interfaces y clases abstractas

Tema 05 · Herencia: relación y límites

Objetivo

Usar extends para especializar un tipo cuando la relación de sustitución y el contrato lo justifican.

Explicación

Una subclase hereda miembros accesibles de una superclase y puede añadir o redefinir comportamiento. En Java una clase tiene una sola superclase directa; las interfaces ofrecen otra forma de compartir contratos.

Usa herencia cuando un Libro pueda tratarse donde se espera un MaterialBiblioteca sin romper las expectativas del cliente. Compartir unos campos por sí solo no demuestra esa relación. La subclase debe respetar el comportamiento prometido por la base.

Ejemplo guiado

Especializar una clase del dominio · Java

```
class MaterialBiblioteca {
  private final String titulo;
  MaterialBiblioteca(String titulo) { this.titulo = titulo; }
  String titulo() { return titulo; }
}
class Libro extends MaterialBiblioteca {
  Libro(String titulo) { super(titulo); }
}
```

Lectura

extends declara la subclase. super(titulo) llama al constructor de la base; Java no hereda constructores automáticamente. Los campos privados siguen siendo parte de la clase base, no se acceden directamente desde Libro.

Alcance y precisión

No heredes solo para reutilizar código: la subclase queda vinculada al contrato y a futuras decisiones de la base. Comprueba que se pueda sustituir de forma razonable.

Práctica

Decide si Revista debe extender MaterialBiblioteca o contener uno. Explica si representa un subtipo o una relación entre objetos y qué operaciones compartiría.

Comprobación

Una clase extiende como máximo una clase. Si necesitas compartir varias capacidades, revisa interfaces o composición antes de crear una jerarquía artificial.

Fuentes enlazadas: Herencia en Dev.java · Clases e interfaces Java

Tema 06 · Polimorfismo y sobrescritura

Objetivo

Invocar una operación común sobre varios subtipos y distinguir sobrescritura de sobrecarga.

Explicación

Cuando una referencia de tipo base apunta a un objeto de una subclase, una llamada a un método de instancia sobrescrito ejecuta la implementación asociada al objeto real en tiempo de ejecución. Esto permite escribir clientes que operan sobre el tipo base.

La anotación @Override ayuda al compilador a comprobar que se está sobrescribiendo un método existente. Sobrecargar es declarar métodos con el mismo nombre y distintos parámetros; no es el mismo mecanismo que redefinir el comportamiento heredado.

Ejemplo guiado

Despachar según el objeto real · Java

```
abstract class Material {
  abstract String tipo();
}
class Libro extends Material {
  @Override String tipo() { return "libro"; }
}
class Revista extends Material {
  @Override String tipo() { return "revista"; }
}
Material m = new Revista();
String etiqueta = m.tipo(); // "revista"
```

Lectura

La variable m se declara como Material, por eso el cliente depende de la operación tipo(). El objeto creado es Revista y su implementación sobrescrita determina el resultado.

Alcance y precisión

Métodos static no se despachan igual que métodos de instancia sobrescritos: su resolución se asocia a la clase declarada. Este ejemplo solo trata métodos de instancia.

Práctica

Añade Audiolibro y llama tipo() desde una función que reciba Material. No añadas una cadena de if por subtipo al código cliente.

Comprobación

Si m se declara Material, solo puedes llamar miembros visibles en Material; cuál implementación se ejecuta depende del objeto, no del nombre de la variable.

Fuentes enlazadas: Polimorfismo Java · Sobrescritura de métodos

Tema 07 · Interfaces como contratos

Objetivo

Separar lo que un cliente necesita de cómo una clase concreta lo proporciona.

Explicación

Una interfaz Java es un tipo de referencia que declara operaciones. Una clase que la implementa promete ofrecer esos métodos; el cliente puede depender de la interfaz sin conocer la implementación concreta.

Las interfaces también pueden declarar métodos default y static con reglas propias. Para el ejercicio básico, usa la forma mínima: contrato, implementación y una llamada por la interfaz.

Ejemplo guiado

Un contrato de identificación · Java

```
interface Identificable {
  String id();
}
class Libro implements Identificable {
  private final String isbn;
```

```
  Libro(String isbn) { this.isbn = isbn; }
  @Override public String id() { return isbn; }
}
Identificable item = new Libro("978-1");
```

Lectura

En esta interfaz, id() no declara modificadores ni cuerpo, así que es implícitamente public abstract. La implementación debe declarar public para no reducir la visibilidad. item permite usar id() sin exigir que el cliente conozca Libro.

Alcance y precisión

Implementar una interfaz no obliga a usar un contenedor de dependencias. Java admite que una clase implemente varias interfaces, aunque solo extienda una clase.

Práctica

Define una interfaz RepositorioLibros con buscarPorIsbn. Implementa una versión en memoria y otra alternativa simulada; haz que el servicio hable con el contrato.

Comprobación

El contrato debe expresar lo que el cliente necesita, no todos los métodos internos de la implementación.

Fuentes enlazadas: Interfaces en Java · Dev.java · Implementar una interfaz

Tema 08 · Composición y delegación

Objetivo

Modelar relaciones entre objetos mediante campos y delegación cuando no exista un subtipo real.

Explicación

Componer es mantener una referencia a otro objeto y coordinarse con él. Si Biblioteca contiene un Catalogo, eso expresa una relación tiene-un; no hace falta afirmar que Biblioteca es un tipo de Catalogo.

La composición facilita cambiar el colaborador sin crear una jerarquía. No es una regla absoluta contra la herencia: usa cada mecanismo para la relación que representa y considera el ciclo de vida y la propiedad de los objetos.

Ejemplo guiado

Una biblioteca delega la búsqueda al catálogo · Java

```
import java.util.List;
class Catalogo {
  List<Libro> buscar(String texto) { return List.of(); }
}
class Biblioteca {
  private final Catalogo catalogo;
  Biblioteca(Catalogo catalogo) { this.catalogo = catalogo; }
  List<Libro> buscar(String q) { return catalogo.buscar(q); }
}
```

Lectura

Biblioteca no extiende Catalogo. Conserva una referencia y delega la operación. List.of() es solo un resultado vacío de ejemplo para que el fragmento compile; una búsqueda real tendría que consultar los datos.

Alcance y precisión

El ejemplo usa List.of(), disponible desde Java 9. Si tu proyecto usa una versión anterior, sustituye la lista vacía por una colección compatible.

Práctica

Añade un ServicioPrestamos que reciba una interfaz de catálogo en su constructor. Explica qué implementación podrías sustituir sin cambiar la lógica del servicio.

Comprobación

Usa extends si el tipo derivado respeta el contrato del tipo base; usa un campo y delegación si el objeto colabora con otro objeto.

Fuentes enlazadas: Objetos y clases · Herencia y límites de Java

Tema 09 · SRP: responsabilidad única

Objetivo

Separar código que cambia por motivos distintos sin convertir el principio en una regla de un método por clase.

Explicación

El Single Responsibility Principle (SRP) es una guía de diseño, no una regla de Java. Robert C. Martin lo formula alrededor de agrupar lo que cambia por las mismas razones y separar lo que cambia por motivos diferentes.

Una clase puede tener varios métodos si todos responden al mismo conjunto de responsabilidades. El ejemplo separa la creación de un préstamo de su representación CSV: el cambio en un formato de exportación no obliga por sí mismo a modificar la regla que crea el préstamo.

Ejemplo guiado

Dos motivos de cambio, dos colaboradores · Java

```
class Prestamo {
  private final String isbn;
  Prestamo(String isbn) { this.isbn = isbn; }
  String isbn() { return isbn; }
}
class ServicioPrestamos {
  Prestamo crear(String isbn) { return new Prestamo(isbn); }
}
class ExportadorCsv {
  String fila(Prestamo p) { return p.isbn(); }
}
```

Lectura

El CSV mostrado es intencionadamente mínimo: representa el dato central y no una especificación completa de escapado CSV. La idea es identificar dos actores y dos motivos de cambio.

Alcance y precisión

SRP no exige una clase por método ni predice una estructura universal. Revisa cohesión, acoplamiento, tamaño del cambio y quién solicita esa modificación.

Práctica

Si Prestamo calcula una multa, guarda en una base de datos y prepara una pantalla, identifica quién pide cada cambio y decide si una responsabilidad debe salir de la entidad.

Comprobación

No dividas solo por contar métodos. La separación se justifica cuando hay razones de cambio distintas y el nuevo límite mejora claridad sin crear capas vacías.

Fuentes enlazadas: Robert C. Martin · SRP · Java: estado y comportamiento

Tema 10 · OCP: extender sin dispersar cambios

Objetivo

Usar una abstracción cuando existe una variación esperada y mantener estable el código que coordina esa variación.

Explicación

El Open-Closed Principle es una guía de diseño: ciertos módulos pueden quedar abiertos a extensiones previsibles y cerrados a cambios repetidos en su núcleo. No significa que el código nunca deba modificarse ni que haya que crear una interfaz para cada clase.

Aquí ServicioRenovacion delega la decisión a una política. Añadir una nueva política para un caso esperado puede añadir una implementación sin introducir un if por cada tipo dentro del servicio. Si solo existe una regla estable, una abstracción extra puede no aportar valor.

Ejemplo guiado

Una política variable detrás de interfaz · Java

```
interface PoliticaRenovacion {
  boolean permite(Prestamo prestamo);
}
class ServicioRenovacion {
  private final PoliticaRenovacion politica;
  ServicioRenovacion(PoliticaRenovacion p) { politica = p; }
  boolean renovar(Prestamo p) { return politica.permite(p); }
}
```

Lectura

El servicio conoce el contrato, no las reglas concretas. Cada política nueva debe respetar el significado acordado de permite; cambiar la semántica del contrato puede requerir modificar clientes.

Alcance y precisión

OCP es un objetivo estratégico para cambios esperados, no una garantía formal del compilador. Elige qué dimensión conviene estabilizar y revisa el coste de extensión.

Práctica

Añade dos políticas de renovación con condiciones distintas. Mantén la decisión fuera de ServicioRenovacion y escribe el criterio de cada implementación.

Comprobación

Una extensión real añade un comportamiento coherente sin llenar el servicio de casos especiales; las reglas deben quedar comprobables por separado.

Fuentes enlazadas: Robert C. Martin · Open-Closed · Interfaces Java

Tema 11 · Inyección de dependencias

Objetivo

Entregar las colaboraciones a un objeto desde fuera y mantener explícito qué necesita para funcionar.

Explicación

Un objeto tiene una dependencia cuando necesita otro objeto para cumplir su trabajo. Con inyección por constructor, recibe esa dependencia al crearse en lugar de buscarla o construirla internamente. En Java puede hacerse manualmente; un contenedor como Spring automatiza el cableado de sus beans.

Constructorizar dependencias requeridas deja visible el contrato de creación. La abstracción Clock permite sustituir el reloj del sistema por otro controlado en una prueba, sin que el servicio tenga

que consultar el tiempo globalmente.

Ejemplo guiado

Pasar el reloj al servicio · Java

```
import java.time.Clock;
import java.time.LocalDate;
class ServicioPrestamos {
  private final Clock reloj;
  ServicioPrestamos(Clock reloj) { this.reloj = reloj; }
  LocalDate fechaActual() { return LocalDate.now(reloj); }
}
var servicio = new ServicioPrestamos(Clock.systemUTC());
```

Lectura

Clock se entrega por el constructor y el servicio lo conserva. En una prueba se puede proporcionar otra implementación de Clock; el ejemplo de creación usa el reloj UTC de Java.

Alcance y precisión

La última línea usa var (Java 10+). Si tu proyecto usa una versión anterior, declara explícitamente ServicioPrestamos. El ejemplo no necesita Spring; la inyección es una forma de ensamblar objetos.

Práctica

Pasa un repositorio por constructor junto con Clock. Identifica cuáles dependencias son obligatorias y cuáles podrían tener una configuración opcional.

Comprobación

Si el servicio construye un repositorio concreto dentro de cada método, sustituirlo para probar o cambiar implementación se vuelve más difícil.

Fuentes enlazadas: Inyección de dependencias · Spring · API Clock · Java

Tema 12 · Factory y Strategy

Objetivo

Distinguir la creación de un objeto de la estrategia intercambiable que el objeto representa.

Explicación

Strategy encapsula una decisión o algoritmo detrás de un contrato para que el cliente pueda trabajar con varias alternativas. Una fábrica es una operación que decide y devuelve qué objeto crear. Son nombres de patrones de diseño, no palabras clave ni mecanismos especiales de Java.

El ejemplo devuelve un Formateador y oculta la elección entre dos implementaciones sencillas. Las fábricas estáticas también aparecen en las APIs Java, por ejemplo List.of; el método crear de aquí es una versión mínima para el dominio.

Ejemplo guiado

Elegir una estrategia mediante una fábrica · Java

```
interface Formateador { String aplicar(String texto); }
class FormatoPlano implements Formateador {
  public String aplicar(String s) { return s; }
}
class FormatoMarcado implements Formateador {
  public String aplicar(String s) { return "[" + s + "]"; }
}
class Formateadores {
  static Formateador crear(boolean marcado) {
    return marcado ? new FormatoMarcado() : new FormatoPlano();
  }
}
```

Lectura

Formateador es Strategy: las implementaciones son sustituibles desde el cliente. Formateadores.crear centraliza la elección de una instancia; no hace falta una fábrica si la construcción sigue siendo trivial.

Alcance y precisión

Evita convertir un ejemplo de patrón en una receta universal. Compara el coste de la abstracción con el número y la probabilidad de variaciones reales.

Práctica

Añade un formato que muestre el ISBN con un prefijo. Cambia la fábrica para seleccionarlo y verifica que el cliente sigue llamando aplicar(String).

Comprobación

La fábrica decide qué crear; la estrategia define el comportamiento intercambiable. No confundas el contrato de comportamiento con el mecanismo de construcción.

Fuentes enlazadas: Interfaces Java · Fábricas de colecciones Java

Tema 13 · Excepciones y contratos

Objetivo

Comunicar errores excepcionales y especificar qué entradas acepta una operación y qué resultado promete.

Explicación

Una excepción interrumpe el flujo normal y transporta información sobre un error. Las excepciones checked están sujetas al requisito de capturar o declarar; RuntimeException y Error y sus subclases son unchecked. No conviertas todas las entradas inválidas en una excepción sin pensar en el contrato.

Valida en la frontera adecuada y deja que una capa responsable decida cómo presentar o recuperar el error. No captures una excepción solo para ocultarla; conserva la causa o tradúcela con contexto si la nueva capa puede actuar sobre ella.

Ejemplo guiado

Traducir texto externo a un entero válido · Java

```
int leerDias(String entrada) {
  try {
    return Integer.parseInt(entrada);
  } catch (NumberFormatException causa) {
    throw new IllegalArgumentException("Días inválidos", causa);
  }
}
```

Lectura

parseInt puede lanzar NumberFormatException si el texto no representa un entero. La función comunica al llamador un error de argumento y conserva la causa original para depuración.

Alcance y precisión

La recomendación concreta entre checked y unchecked depende del contexto y de la capacidad real de recuperación. Documenta el comportamiento observable de la API.

Práctica

Define qué ocurre si los días son negativos. ¿Es inválido el argumento, un resultado permitido o un error de estado? Implementa y escribe esa decisión en la documentación del método.

Comprobación

Una excepción no es una salida normal para un caso esperado que el llamador debe elegir; para ese caso suele convenir representar el resultado de manera explícita.

Fuentes enlazadas: Excepciones en Java · Dev.java · Excepciones checked · JLS 17 · Integer.parseInt · Java SE 17 API · IllegalArgumentException · Java SE 17 API

Tema 14 · Pruebas unitarias con JUnit

Objetivo

Comprobar un comportamiento observable de una clase con una prueba pequeña y repetible.

Explicación

Una prueba unitaria prepara una condición, invoca una operación y compara el resultado observado con el esperado. JUnit Jupiter ofrece anotaciones y aserciones para escribir esos casos; una prueba no demuestra que un programa entero esté libre de defectos.

El ejemplo comprueba el estado inicial expuesto por Libro, no la visibilidad de sus campos. El manual oficial consultado corresponde a JUnit 6.1.3, que requiere Java 17 o superior en tiempo de ejecución.

Ejemplo guiado

Comprobar el estado inicial de un libro · JUnit 6

```
import static org.junit.jupiter.api.Assertions.assertFalse;
import org.junit.jupiter.api.Test;
class LibroTest {
  @Test
  void comienzaSinPrestamo() {
    Libro libro = new Libro("978-1", "POO");
    assertFalse(libro.estaPrestado());
  }
}
```

Lectura

La anotación @Test identifica el método de prueba; assertFalse falla si la condición no es falsa. LibroTest debe poder acceder a los miembros del ejemplo, por ejemplo al estar en el mismo paquete.

Alcance y precisión

JUnit 6.1.3 requiere Java 17 o superior en tiempo de ejecución. Comprueba qué versión de JUnit admite el JDK y el sistema de construcción de tu proyecto.

Práctica

Añade una prueba que compruebe que prestar un libro cambia el resultado observable de estaPrestado(). Añade otro caso para devolverlo.

Comprobación

Cada prueba debe fallar por una razón útil cuando cambia el comportamiento y pasar cuando se cumple la expectativa que documenta.

Fuentes enlazadas: Escribir pruebas con JUnit Jupiter · 6.1.3 · JUnit 6.1.3 · requisitos Java

Tema 15 · Diseñar un dominio completo

Objetivo

Integrar estado, contratos, colaboraciones, reglas y ejemplos de prueba en un pequeño modelo orientado a objetos.

Explicación

Diseña el dominio desde las reglas que debe respetar, no desde una lista de sustantivos. En una biblioteca de práctica, Libro mantiene su identidad, Prestamo representa la relación temporal, ServicioPrestamos coordina el caso de uso y un repositorio abstrae cómo se guardan los datos.

No todas las clases necesitan herencia, interfaces o un patrón. Introduce abstracciones cuando exista una variación o un límite útil; mantén las reglas cerca del objeto o servicio que puede hacerlas cumplir y prueba resultados observables.

Ejemplo guiado

Un servicio depende del contrato de almacenamiento · Java

```
interface RepositorioPrestamos {
  void guardar(Prestamo prestamo);
}
class ServicioPrestamos {
  private final RepositorioPrestamos repositorio;
  ServicioPrestamos(RepositorioPrestamos r) { repositorio = r; }
  void registrar(Prestamo p) { repositorio.guardar(p); }
}
```

Lectura

El servicio expresa la operación de aplicación y recibe el colaborador al construirse. La implementación concreta del repositorio queda fuera del fragmento para centrar el ejercicio en el límite entre política y almacenamiento.

Alcance y precisión

Este esqueleto no es una arquitectura obligatoria ni un sistema de producción: faltan persistencia, validación, concurrencia y reglas específicas de la organización.

Práctica

Entrega un diagrama de clases y una prueba para registrar un préstamo. Implementa una invariante, una colaboración por interfaz y pruebas del caso normal, límite y error; explica qué decisión deliberadamente no modelaste.

Comprobación

Puedes explicar quién posee cada dato, qué regla protege cada clase, qué interfaz necesita el cliente y qué resultado o límite comprueba cada prueba.

Fuentes enlazadas: POO en Java · Dev.java · Interfaces Java · Escribir pruebas con JUnit Jupiter · 6.1.3

Proyecto integrador · préstamos de biblioteca

Objetivo

Modela el préstamo de ejemplares de una biblioteca de centro. El diseño debe proteger las reglas del dominio y permitir probarlo sin depender de una pantalla o una base de datos.

Pasos de trabajo

1. Define Socio, Libro, Ejemplar y Prestamo. Separa el título bibliográfico del ejemplar físico que puede prestarse.

2. Crea un servicio que abra un préstamo solo si el ejemplar está disponible y el socio puede pedirlo.

3. Añade una operación de devolución que cierre un préstamo abierto y rechace una segunda devolución.

4. Usa una interfaz de repositorio para desacoplar el servicio de la colección en memoria y escribe pruebas JUnit para las reglas.

Comprobaciones

- Un ejemplar no puede tener dos préstamos abiertos a la vez.

- La devolución conserva el historial y registra una fecha coherente.

- Las pruebas cubren disponibilidad, rechazo y devolución repetida.

Casos mínimos de prueba

- Ejemplar disponible y socio apto: se abre el préstamo y el estado observable cambia.

- Ejemplar ya prestado: se rechaza la segunda solicitud y el préstamo anterior permanece.

- Préstamo activo: la devolución lo cierra y conserva el historial.

- Préstamo cerrado: repetir la devolución sigue la regla acordada sin alterar el registro.

Entrega esperada

Entrega un diagrama de responsabilidades, la implementación y las pruebas de los casos habituales y límite. Explica por qué el servicio depende de RepositorioPrestamos y ejecuta la misma batería contra la colección en memoria y contra una segunda implementación de prueba.

Explicación final

Explica qué clase es responsable de cada regla, qué colaboración resolviste mediante composición y qué cambiarías si mañana el almacenamiento dejara de ser una lista en memoria.

Fuentes enlazadas: Programación orientada a objetos · Dev.java · Interfaces Java · Dev.java · Pruebas con JUnit Jupiter 6.1.3